

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de guión.

Contenido multimedia

Definición de multimedia. Tipos de recursos multimedia

Inclusión de contenido multimedia en páginas web

Gráficos multimedia

Definición de multimedia. Tipos de recursos multimedia

Definición de Multimedia

- **Multimedia** combina texto, imágenes, audio, vídeo y animaciones para comunicar ideas de forma efectiva.
- En el contexto de las páginas web, el multimedia enriquece la experiencia del usuario (UX) al hacerlo más interactivo y visualmente atractivo.

Tipos de recursos multimedia

1. **Texto:** Base de cualquier contenido web, como encabezados, párrafos y subtítulos.
2. **Imágenes:** Fotografías, iconos, gráficos vectoriales (SVG), infografías.
3. **Audio:** Clips de sonido, música de fondo, podcasts.
4. **Vídeo:** Clips, tutoriales, presentaciones, transmisiones en vivo.
5. **Animaciones:** GIFs, transiciones interactivas, banners.
6. **Elementos interactivos:** Mapas interactivos, sliders, formularios dinámicos.

Ejemplo práctico: Un sitio web como <https://www.nationalgeographic.com/> que utiliza texto, imágenes, vídeos y animaciones para ofrecer una experiencia rica en multimedia.

Inclusión de contenido multimedia en páginas web

Adaptadores para recursos multimedia

- HTML5 permite incorporar multimedia de manera nativa usando etiquetas como <audio> y <video>, sin necesidad de plugins.
- Frameworks o librerías como **Plyr.js** para reproductores multimedia personalizables.

Enlace a diferentes recursos desde páginas web

- Enlace externo: Usar etiquetas <a> para enlazar a multimedia alojado en servidores externos.

html

```
<a href="https://example.com/video.mp4" target="_blank">Ver vídeo</a>
```

Incrustación de contenido multimedia

- Imágenes:

```

```

- Vídeos incrustados (ejemplo de YouTube):

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/video_id" frameborder="0" allowfullscreen></iframe>
```

- Vídeo nativo:

```
<video width="560" height="315" controls>  
  <source src="video.mp4" type="video/mp4">  
  Tu navegador no soporta vídeos. Descárgalo <a href="video.mp4">aquí</a>.  
</video>
```

Atributos útiles:

- controls: Muestra los controles de reproducción.
- autoplay: Reproduce automáticamente el vídeo (¡cuidado con la experiencia del usuario!).
- loop: Reproduce el vídeo en bucle.

- Audio nativo:

```
<audio controls>  
  <source src="audio.mp3" type="audio/mpeg">  
  Tu navegador no soporta audio.  
</audio>
```

Formatos de archivo web. El estándar MIME

- **MIME (Multipurpose Internet Mail Extensions):**

Define cómo los navegadores interpretan diferentes formatos de archivo.

- Ejemplo: image/jpeg, audio/mpeg, video/mp4.

Tipo de contenido	Extensión común	MIME Type
Imagen	.jpeg, .png	image/jpeg
Audio	.mp3, .ogg	audio/mpeg
Vídeo	.mp4, .webm	video/mp4

Tipos de reproducción: Streaming y carga progresiva

- **Streaming:** El contenido se reproduce a medida que se descarga (e.g., YouTube, Spotify).
- **Carga progresiva:** Se descarga completamente antes de la reproducción (e.g., descargar un MP3).

Comparativa del tratamiento de contenido multimedia en versiones de HTML

- HTML4 requería plugins como Flash para multimedia.
- HTML5 permite inclusión directa con <audio> y <video>, ofreciendo mayor compatibilidad y accesibilidad.

Ejemplo práctico: Crear una página HTML con un vídeo <video> y controles personalizados usando CSS.

Gráficos multimedia

Formatos gráficos: Comparativa

- Formatos más usados:
 - **JPEG**: Fotos, buena compresión con pérdida.
 - **PNG**: Imágenes sin fondo, compresión sin pérdida.
 - **SVG**: Gráficos vectoriales, escalables sin pérdida de calidad.
 - **GIF**: Imágenes animadas con paleta limitada.
 - **WEBP**: Formato moderno con mejor compresión.

Formato	Uso recomendado	Pros	Contras
JPEG	Fotos	Alta compatibilidad	Pérdida de calidad al comprimir
PNG	Logos, transparencias	Transparencias perfectas	Tamaño mayor que JPEG
SVG	Iconos, gráficos vectoriales	Escalable, ligero	No ideal para fotos
GIF	Animaciones simples	Fácil de implementar	Limitado a 256 colores

Repositorios de imágenes

- **Gratuitos**: Unsplash, Pexels, Pixabay.
- **De pago**: Shutterstock, Adobe Stock.

Ejemplo práctico: Inserta una imagen SVG y prueba a cambiar el tamaño para mostrar cómo se escala perfectamente.

Tipos de gráficos

1. **Fotografías:** Usadas para capturar momentos realistas.
2. **Imágenes vectorizadas:** Para iconos y logos escalables.
3. **Iconos:** Elementos gráficos pequeños y representativos (pueden ser SVG o PNG).

Herramientas para el tratamiento gráfico

- **Edición básica:** Canva, Figma.
- **Edición avanzada:** Photoshop, GIMP.
- **Filtros y efectos:** Aplicación de filtros CSS para cambios rápidos:

CSS:

```
img {  
    filter: grayscale(50%) blur(2px);  
}
```

Tabla con los filtros CSS

Filtro	Descripción	Ejemplo de uso
blur()	Aplica un desenfoque a la imagen. El valor es en píxeles (px).	<code>filter: blur(5px);</code>
brightness()	Ajusta el brillo. Valores mayores a 1 aumentan el brillo, menores a 1 lo reducen.	<code>filter: brightness(1.5);</code>
contrast()	Ajusta el contraste. Valores mayores a 1 aumentan el contraste, menores a 1 lo reducen.	<code>filter: contrast(2);</code>

Filtro	Descripción	Ejemplo de uso
grayscale()	Convierte la imagen a escala de grises. El valor es un porcentaje (0% a 100%) o decimal (0 a 1).	<code>filter: grayscale(50%);</code>
hue-rotate()	Rota los colores de la imagen según el círculo cromático. El valor es en grados (deg).	<code>filter: hue-rotate(90deg);</code>
invert()	Invierte los colores de la imagen. El valor es un porcentaje (0% a 100%) o decimal (0 a 1).	<code>filter: invert(100%);</code>
opacity()	Ajusta la opacidad de la imagen. El valor es un porcentaje (0% a 100%) o decimal (0 a 1).	<code>filter: opacity(75%);</code>
saturate()	Ajusta la saturación. Valores mayores a 1 aumentan la saturación, menores a 1 la reducen.	<code>filter: saturate(2);</code>
sepia()	Aplica un efecto sepia. El valor es un porcentaje (0% a 100%) o decimal (0 a 1).	<code>filter: sepia(50%);</code>
drop-shadow()	Aplica una sombra a la imagen. Similar a box-shadow, pero específica para imágenes y contenido SVG.	<code>filter: drop-shadow(10px 10px 5px rgba(0, 0, 0, 0.5));</code>

Conversión de formatos gráficos

- Conversión directa en línea: TinyPNG, CloudConvert.
- Alternativa con herramientas locales:
 - Usar Photoshop o GIMP para guardar en diferentes formatos.

Ejemplo práctico: Edita una imagen utilizando filtros CSS y cambia entre formatos PNG, JPEG y SVG.

Uso de <picture> y srcset en HTML5

El elemento <picture> permite especificar múltiples fuentes de imagen para un mismo elemento, lo que resulta útil para servir diferentes imágenes en función del dispositivo, la resolución de pantalla, u otros criterios. Esto ayuda a optimizar la carga de imágenes y a mejorar el rendimiento de las páginas web.

• Ejemplo básico con <picture>

```
<picture>
  <!-- Imagen para pantallas anchas -->
  <source media="(min-width: 800px)" srcset="imagen-grande.jpg">
  <!-- Imagen para pantallas medianas -->
  <source media="(min-width: 480px)" srcset="imagen-mediana.jpg">
  <!-- Imagen predeterminada para pantallas pequeñas -->
  
</picture>
```

• Cómo funciona este ejemplo

1. El navegador selecciona la fuente de imagen adecuada según las condiciones especificadas en los atributos media de las etiquetas <source>.
2. Si la pantalla tiene un ancho:
3. Mayor o igual a 800px, se cargará imagen-grande.jpg.

4. Entre 480px y 799px, se cargará imagen-mediana.jpg.
5. Menor a 480px, se cargará imagen-pequeña.jpg especificada en la etiqueta como predeterminada.

Uso del atributo srcset

El atributo srcset permite definir varias versiones de una misma imagen con diferentes resoluciones o tamaños. El navegador selecciona automáticamente la imagen más adecuada según la densidad de píxeles del dispositivo (DPR) o el tamaño del área de visualización.

• Ejemplo básico con srcset

```

```

• Cómo funciona este ejemplo

1. srcset: Define diferentes versiones de la misma imagen:
 - imagen-1x.jpg: Para pantallas estándar.
 - imagen-2x.jpg: Para pantallas con densidad 2x (como Retina Display).
 - imagen-3x.jpg: Para pantallas con densidad 3x.
2. sizes: Especifica el ancho que ocupará la imagen en diferentes condiciones:
 - Si el ancho del dispositivo es menor o igual a 600px, la imagen ocupará el 100% del ancho del viewport.
 - En otros casos, ocupará el 50% del ancho del viewport.

Ventajas de <picture> y srcset

1. **Rendimiento optimizado:** Carga la imagen más adecuada según el dispositivo, reduciendo el tamaño de los datos transferidos.
2. **Mejora de la experiencia del usuario:** Las imágenes se ajustan al dispositivo, asegurando una mejor calidad y velocidad de carga.

3. **Flexibilidad:** Puedes usar diferentes formatos de imagen como WebP para navegadores modernos y PNG/JPEG como respaldo.

Comparación de <picture> y srcset

Característica	<picture>	srcset
Uso principal	Para cargar diferentes versiones de una imagen en función de media queries o formatos.	Para cargar imágenes según la resolución del dispositivo o el ancho de la pantalla.
Nivel de control	Mayor control, ideal para manejar diferentes formatos (JPEG, WebP, etc.) y condiciones.	Más sencillo, ideal para manejar resoluciones.
Soporte de formatos	Permite alternar entre formatos de imagen (WebP, PNG, etc.).	Solo define imágenes de diferentes resoluciones.
Compatibilidad	Totalmente soportado en navegadores modernos.	También es ampliamente soportado.

Ejemplo combinado con <picture> y srcset:

html

```
<picture>
  <!-- Imagen para pantallas de alta resolución (retina o 4K) -->
  <source srcset="img/grande-2x.jpg 2x, img/grande-1x.jpg 1x" media="(min-
width: 1200px)">

  <!-- Imagen para pantallas medianas -->
  <source srcset="img/mediana-2x.jpg 2x, img/mediana-1x.jpg 1x" media="(min-
width: 768px)">

  <!-- Imagen para pantallas pequeñas -->
  <source srcset="img/pequena-2x.jpg 2x, img/pequena-1x.jpg 1x" media="(max-
width: 767px)">

  <!-- Fallback para navegadores antiguos que no soportan <picture> -->
  
</picture>
```

Desglose del ejemplo:

1. **<picture>:**

- Define un contenedor para diferentes versiones de una imagen.
- Contiene varios elementos `<source>` y un `<img>` como fallback.

2. **<source>:**

- Usa `srcset` para definir imágenes con diferentes resoluciones.
- Usa `media` para definir qué imagen debe usarse según el ancho del viewport.

3. **srcset:**

- Ofrece opciones de imagen dependiendo de la densidad de píxeles del dispositivo:
- 2x: Imagen para pantallas de alta densidad (retina).
- 1x: Imagen estándar.

4. **media:**

- Funciona como una media query en CSS.
- Define reglas como `(min-width: 1200px)` o `(max-width: 767px)` para servir la imagen adecuada según el ancho del viewport.

5. ** como fallback:**

- Siempre se debe incluir un `<img>` al final para compatibilidad con navegadores antiguos.

Práctica:

- Creación de una página web sencilla con:
 - Imágenes optimizadas con picture y scrset
 - Un vídeo <video> con controles personalizados.
 - Un audio con un reproductor <audio>.